A Practitioner's Guide to Deep Learning for Individual Heterogeneity

February 2026

Abstract

This paper provides a step-by-step guide to implementing the deep learning framework for individual heterogeneity developed in Farrell et al. (2021) and Farrell et al. (2025). We show how neural networks can directly estimate heterogeneous structural parameters from individual-level data, and how influence function corrections deliver valid confidence intervals despite the regularization bias inherent in neural network estimation. Our running application uses H&M fashion transaction data: we train two-tower contrastive embeddings to represent consumer preferences, estimate a multinomial choice model with heterogeneous price sensitivity, and construct influence-function-corrected confidence intervals for average elasticities and counterfactual pricing experiments. We provide complete, documented code and a companion `Python` package (`deep-inference`) that handles cross-fitting, Lambda estimation, and influence function assembly automatically. The guide is intended for applied researchers and practitioners who want to move beyond assuming homogeneous effects without sacrificing valid inference.

Keywords: deep learning, individual heterogeneity, influence functions, multinomial choice, demand estimation, contrastive learning

JEL Codes: C14, C45, D12, L81

Contents

1 Introduction

Individual heterogeneity is ubiquitous in economic data. Consumers differ in their sensitivity to prices, workers respond differently to job training, and patients vary in their treatment effects. Empirical researchers have long recognized this heterogeneity and built it into their models, from discrete choice models (McFadden, 1974; Train, 2009) to personalized pricing (Dubé and Misra, 2023) and heterogeneous treatment effects more broadly. But estimating heterogeneous parameters at the individual level, and doing valid statistical inference on the resulting objects, has remained a difficult task. Traditional approaches typically assume a parametric distribution for heterogeneity (e.g., normal random coefficients) or restrict attention to a small number of discrete types.

This paper is a practitioner's guide to a different approach. Farrell et al. (2021) and Farrell et al. (2025) show how deep neural networks can directly estimate individual-level structural parameters $\theta(x)$ from observational data, and how influence function corrections deliver valid confidence intervals despite the regularization bias inherent in neural network estimation. The theoretical foundations are in their *Econometrica* papers. Our goal here is to make the method accessible and practical.

We walk through the entire pipeline, from data preparation to model specification to inference to counterfactuals, using a running example that illustrates every step. We provide complete code, a companion Python package (`deep-inference`), and a simulation study validating the method.

1.1 The Idea in Brief

Consider a structural model with individual-level parameters. A researcher observes outcomes Y_i, treatments or product characteristics T_i, and individual covariates X_i for $i = 1, \ldots, n$ observations. The structural model specifies a loss function $\ell(Y_i, T_i, \theta)$, and the individual's true parameters $\theta^*(X_i)$ minimize the expected loss conditional on their type X_i.

The key insight of FLM is: let a neural network directly output the structural parameters. Train a network $\hat{\theta}(X_i)$ by minimizing the structural loss across observations. Then use influence functions to correct for the regularization bias that neural networks inevitably introduce.

The framework applies broadly. For example:

- In a *logit* demand model, ℓ is the binary cross-entropy and $\theta(X_i) = (\alpha_i, \beta_i)$ captures individual-level intercept and price sensitivity.
- In a *Poisson* count model for store visits, ℓ is the Poisson negative log-likelihood and β_i is the individual's response to a promotional campaign.
- In a *multinomial choice* model, ℓ is the softmax cross-entropy and $\theta(X_i)$ is a vector of taste parameters across product attributes.

The target $\mu^* = E[H(X, \theta^*(X), \tilde{t})]$ can be an average parameter ($H = \theta_k$, e.g., mean price sensitivity), a marginal effect ($H = g'(\theta'\tilde{t}) \cdot \beta$), or an economic quantity like expected revenue ($H = \tilde{t} \cdot g(\theta'\tilde{t})$).

Why is correction necessary? Neural networks use regularization (weight decay, dropout, early stopping) to prevent overfitting. This regularization shrinks estimates toward zero, making naive standard errors, computed from the Hessian of the structural loss, systematically too small. The resulting confidence intervals severely undercover. In our simulations, naive coverage can fall below 20% when the nominal level is 95%.

The influence function (IF) correction accounts for the estimation noise in $\hat{\theta}(X)$. The corrected estimator for a target functional $\mu^* = E[H(X, \theta^*(X), \tilde{t})]$ is:

$$\hat{\psi}_i = H_i - H_{\theta,i} \cdot \hat{\Lambda}_i^{-1} \cdot \ell_{\theta,i} \tag{1}$$

where H_i is the target evaluated at the estimated parameters, $H_{\theta,i}$ is the Jacobian of the target with respect to θ, $\hat{\Lambda}_i$ is the expected Hessian of the loss, and $\ell_{\theta,i}$ is the score (gradient of the loss). The average $\bar{\psi} = n^{-1} \sum_i \hat{\psi}_i$ is the debiased estimator, and its standard error provides valid inference.

1.2 Running Example: H&M Fashion Demand

Our running application uses the H&M Personalized Fashion Recommendations dataset (H&M Group, 2022), a publicly available dataset of 1.4 million fashion transactions. We estimate a multinomial choice model for online dress purchases with heterogeneous price sensitivity across consumers.

The application demonstrates the full pipeline:

1. *Representation learning.* We train two-tower contrastive embeddings (van den Oord et al., 2018) to learn dense representations of consumers and products from purchase histories and visual features. Consumer embeddings capture latent taste dimensions; item embeddings capture visual and categorical product attributes.

2. *Choice model estimation.* Consumer embeddings become the covariates X that drive heterogeneity. Each consumer has their own price sensitivity $\theta_{\text{price}}(X_i)$ and taste parameters $\theta_{\text{style},k}(X_i)$. The neural network maps the 64-dimensional consumer embedding to these structural parameters.

3. *Influence function inference.* The `deep-inference` package handles cross-fitting, Lambda estimation, influence function assembly, and confidence interval computation. With a five-line custom loss function, the entire inference pipeline runs automatically.

4. *Counterfactual analysis.* We compute average price elasticities with valid confidence intervals and simulate the revenue impact of a 10% price increase, with uncertainty bands that properly account for the neural network estimation step.

1.3 What This Guide Covers

Section 2 develops the framework: the structural model, target functionals, why naive inference fails, the IF correction, and the three Lambda estimation regimes. Section 3 covers estimation in practice: data requirements, the algorithm step by step, cross-fitting, Lambda estimation, network architecture, diagnostics, and common pitfalls. Section 4 presents the full H&M application with results. Section 5 describes how to validate each component of the pipeline on simulated data where the ground truth is known. Section 6 discusses extensions and other structural models. Appendix A provides annotated code and an installation guide.

Throughout, we emphasize the *practitioner's* perspective: what choices must be made, what can go wrong, and what to look for in the output.

2 The Framework

This section lays out the FLM framework as a practitioner would encounter it: we describe the structural model, define target functionals, explain why naive neural network inference fails, present the influence function correction, and classify problems into three "regimes" that determine how a key quantity, the expected Hessian $\Lambda(x)$, is handled.

2.1 Setup

We observe n i.i.d. observations (Y_i, T_i, X_i) where:

- Y_i is the outcome (chosen product, sales quantity, duration, etc.)
- T_i is the treatment or "action" variable (price, product attributes, dosage)
- $X_i \in \mathbb{R}^{d_x}$ are individual covariates that drive heterogeneity

A *structural model* specifies a loss function $\ell(y, t, \theta)$ parameterized by structural parameters $\theta \in \mathbb{R}^{d_\theta}$. The true parameters for individual i solve:

$$\theta^*(X_i) = \arg\min_{\theta} \; E[\ell(Y, T, \theta) \mid X = X_i] \tag{2}$$

Concrete example. In our H&M application:

- $Y_i \in \{0, 1, \ldots, J-1\}$ is the chosen product from a set of J alternatives
- $T_i \in \mathbb{R}^{J \times K}$ contains the attributes of each alternative (log-price, style embeddings)
- $X_i \in \mathbb{R}^{64}$ is the consumer's learned embedding
- $\theta^*(X_i) = (\beta_{\text{price}}(X_i), \beta_{\text{style},1}(X_i), \ldots, \beta_{\text{style},5}(X_i))$ are the consumer's taste parameters

- The loss is the multinomial logit negative log-likelihood:

$$\ell(y, t, \theta) = -V_y + \log \sum_{j=0}^{J-1} e^{V_j}, \quad V_j = x_j'\theta \tag{3}$$

The framework is *not* limited to discrete choice. Table 1 lists the structural models currently available in the `deep-inference` package, along with their loss functions and parameter dimensions.

Table 1: Available structural models in the `deep-inference` package.

Model	**Loss** $\ell(y, t, \theta)$	**Link**	d_θ
Linear	$(y - \alpha - \beta t)^2$	Identity	2
Logit	$\log(1 + e^\eta) - y\eta$	Logistic	2
Poisson	$e^\eta - y\eta$	Log	2
Gamma	$y/\mu + \log \mu$	Log	2
Multinomial	$-V_y + \log \sum_j e^{V_j}$	Softmax	$(J-1)+K$
Custom	Any differentiable $\ell(y, t, \theta)$	Any	User-specified

Note: $\eta = \alpha + \beta t$, $\mu = g^{-1}(\eta)$. The package includes 13 families total (Weibull, Gumbel, Tobit, NegBin, Probit, Beta, ZIP). Custom losses can be supplied via the `loss=` argument.

2.2 Target Functionals

We are usually not interested in $\theta^*(X_i)$ for every individual, but rather in some summary: an *average* of a function of θ^* across the population:

$$\mu^* = E[H(X, \theta^*(X), \tilde{t})] \tag{4}$$

where H is the *target function* and $\tilde{t}$ is an evaluation point. Common targets include:

Table 2: Target functionals for inference.

Target	$H(x, \theta, \tilde{t})$	**Interpretation**
Average parameter	θ_k	$E[\beta_{\text{price}}]$: mean price sensitivity
Average marginal effect	$g'(\alpha + \beta\tilde{t}) \cdot \beta$	Mean marginal effect at $\tilde{t}$
Elasticity	$\beta \cdot \tilde{t} \cdot (1 - p)$	Price elasticity of demand
Dose-response	$g(\alpha + \beta\tilde{t})$	Predicted outcome at $\tilde{t}$
Profit	$\tilde{t} \cdot g(\alpha + \beta\tilde{t})$	Expected revenue
Custom	Any $H(x, \theta, \tilde{t})$	User-specified

The Jacobian $H_\theta = \partial H / \partial \theta$ is required for the IF correction. For built-in targets, closed-form Jacobians are provided. For custom targets, the package computes them via automatic differentiation.

2.3 Why Naive Inference Fails

To estimate μ^*, we train a neural network $\hat{\theta}(X)$ by minimizing $\sum_i \ell(Y_i, T_i, \hat{\theta}(X_i))$ with sample splitting and then compute $\hat{\mu} = n^{-1} \sum_i H(X_i, \hat{\theta}(X_i), \tilde{t})$.

The *naive* standard error treats $\hat{\theta}$ as if it were the truth and computes $\text{SE}_{\text{naive}} = \text{sd}(H_i)/\sqrt{n}$. This is wrong because $\hat{\theta}$ is itself estimated with error, and neural network regularization introduces systematic bias.

The problem is quantitatively severe. In our simulations (Section 4 and Appendix A):

- Naive 95% CIs achieve only 0–20% actual coverage
- The naive SE is 3–10× too small
- The bias is toward zero (regularization shrinks $\hat{\theta}$)

This is not a small-sample problem; it persists at $n = 50{,}000$. The regularization bias is a *feature* of neural network training (it prevents overfitting) but becomes a *bug* for inference.

To see why formally, consider the decomposition of the naive estimator's error:

$$\sqrt{n}(\hat{\mu}_{\text{naive}} - \mu^*) = \underbrace{\frac{1}{\sqrt{n}} \sum_{i=1}^{n} [H_i - \mu^*]}_{\text{sampling noise}} + \underbrace{\frac{1}{\sqrt{n}} \sum_{i=1}^{n} H_{\theta,i}\left(\hat{\theta}_i - \theta_i^*\right)}_{\text{regularization bias}} + o_P(1) \tag{5}$$

The first term is standard sampling variability and vanishes at rate $\sqrt{n}$. The second term captures the bias introduced by the neural network's regularization: because $\hat{\theta}$ is systematically shrunk toward zero (by weight decay, dropout, and early stopping), $\hat{\theta}_i - \theta_i^*$ is not mean-zero and this term does not vanish.[1] The IF correction in the next section is designed precisely to cancel this second term.

2.4 The Influence Function Correction

The FLM influence function correction removes the regularization bias. For each observation i, define:

$$\psi_i = H_i - H_{\theta,i} \cdot \Lambda_i^{-1} \cdot \ell_{\theta,i} \tag{6}$$

where:

- $H_i = H(X_i, \hat{\theta}(X_i), \tilde{t})$ is the target at estimated parameters
- $H_{\theta,i} = \frac{\partial H}{\partial \theta}\big|_{\hat{\theta}(X_i)} \in \mathbb{R}^{1 \times d_\theta}$ is the target Jacobian
- $\Lambda_i = E[\nabla_\theta^2 \ell(Y, T, \theta) \mid X = X_i]\big|_{\hat{\theta}(X_i)} \in \mathbb{R}^{d_\theta \times d_\theta}$ is the expected Hessian
- $\ell_{\theta,i} = \nabla_\theta \ell(Y_i, T_i, \hat{\theta}(X_i)) \in \mathbb{R}^{d_\theta}$ is the score

[1] In our validation suite, naive 95% CIs achieve 0–20% coverage; IF-corrected CIs achieve 96% ($M = 50$, $n = 5{,}000$, binary logit, eval_06).

The debiased estimator and its standard error are:

$$\hat{\mu} = \frac{1}{n}\sum_{i=1}^{n}\psi_i, \qquad \widehat{\mathrm{SE}} = \frac{\mathrm{sd}(\psi_1, \ldots, \psi_n)}{\sqrt{n}} \tag{7}$$

Intuition. The correction term $H_\theta \Lambda^{-1} \ell_\theta$ is an "adjustment for what the neural net got wrong." The score ℓ_θ measures how far observation i's data is from the fitted model. The Hessian Λ converts this into parameter-space error. The Jacobian H_θ maps this to the target. When the neural net fits perfectly (scores near zero), the correction vanishes. When it makes systematic errors (regularization bias), the correction kicks in.

Neyman orthogonality. The IF ψ in Equation (6) satisfies the *Neyman orthogonality* condition:

$$\frac{\partial}{\partial \theta} E[\psi(Y, T, \theta, \Lambda)]\Big|_{\theta=\theta^*, \Lambda=\Lambda^*} = 0 \tag{8}$$

The correction term $H_\theta \Lambda^{-1} \ell_\theta$ is constructed precisely so that first-order perturbations in $\hat{\theta}$ around θ^* cancel. This is why the debiased estimator $\hat{\mu} = \bar{\psi}$ achieves $\sqrt{n}$-consistency despite the neural network's slow convergence rate: the estimation error in $\hat{\theta}$ enters only through a second-order remainder, which is negligible at the $\sqrt{n}$ scale.[2]

Connection to classical statistics. Equation (6) is a semiparametric influence function in the tradition of Hampel (1974) and Newey and McFadden (1994). The specific form arises from the "one-step correction" or "debiasing" approach used widely in semiparametric econometrics (Chernozhukov et al., 2018). The contribution of FLM is showing that this correction remains valid when the first-stage nuisance (the structural parameters) is estimated by a deep neural network.

2.5 Three Regimes for Lambda

The expected Hessian $\Lambda(X) = E[\nabla^2_\theta \ell(Y, T, \theta) \mid X]$ depends on how the model and data are structured. Three cases arise:

Table 3: Three Lambda estimation regimes.

Regime	When	Lambda method	Cross-fitting
A	RCT, known F_T	Compute (MC integration)	2-way
B	Linear model	Analytic (closed-form)	2-way
C	Obs. + nonlinear	Estimate (ridge)	3-way

[2]Package autodiff matches closed-form oracle scores and Hessians to machine precision ($< 10^{-15}$) on the multinomial logit (eval_02), ensuring that the Neyman orthogonality condition is satisfied numerically.

Regime A: Randomized experiment with known treatment distribution. If T is randomly assigned with known distribution F_T, and the Hessian does not depend on Y, then $\Lambda(X)$ can be computed via Monte Carlo integration: $\Lambda(X) = \int \nabla^2_\theta \ell(y, t, \theta)\, dF_T(t)$. This is the simplest case. Examples: A/B tests, randomized pricing experiments.

Regime B: Linear model. For linear models, the Hessian is constant ($\nabla^2_\theta \ell = 2$), so $\Lambda(X) = 2E[TT' \mid X]$, which can be estimated analytically. This avoids the need for three-way splitting.

Regime C: Observational data with nonlinear model. In most applied settings, including our H&M application, the Hessian depends on θ (e.g., through $p(1-p)$ in logit or softmax probabilities in multinomial logit). Since θ is estimated, we cannot simply compute Λ; we must *estimate* it. FLM propose regressing the individual Hessians on X using a ridge regression. This requires a *three-way* sample split: one part for estimating $\hat{\theta}$, one part for estimating $\hat{\Lambda}$, and one part for assembling the IF. The package handles this automatically.

2.6 The Multinomial Choice Model

Our application uses a multinomial logit (conditional logit) model (McFadden, 1974; Train, 2009). Consumer i chooses from J alternatives, each described by K attributes $x_{ij} \in \mathbb{R}^K$. Utilities are:

$$V_{ij} = x'_{ij}\theta(X_i), \qquad P(Y_i = j \mid X_i) = \frac{e^{V_{ij}}}{\sum_{m=1}^{J} e^{V_{im}}} \tag{9}$$

The FLM framework estimates $\theta(X_i)$ for each individual using individual-level data and neural networks, rather than assuming a parametric distribution for heterogeneity.

The connection between two-tower embeddings and the FLM framework is natural: the consumer embedding X_i captures the latent heterogeneity dimensions, and the neural network maps $X_i \to \theta(X_i)$. The item attributes T_i include prices and (PCA-reduced) item embeddings.

2.7 Available Models and Targets

The `deep-inference` package provides 13 built-in model families (Table 1) and 10+ target functionals (Table 2). For models not in the built-in library, users can supply a custom loss function:

```
import torch
from deep_inference import inference

def my_loss(y, t, theta):
    """Custom structural loss."""
    mu = torch.exp(theta[0] + theta[1] * t)  # log-link
    return mu - y * torch.log(mu)             # Poisson NLL

result = inference(Y, T, X, loss=my_loss, theta_dim=2,
```

```
10                          target_fn=lambda x, th, tt: th[1])
```

The package automatically determines whether the Hessian depends on θ (triggering Regime C and three-way splitting) and computes all required derivatives via automatic differentiation. The user needs only the loss function and the target.

3 Estimation in Practice

This section walks through every step of the estimation procedure, discusses the choices practitioners must make, and flags the pitfalls that can silently invalidate results. For each step, we present the relevant equation from Farrell et al. (2025) alongside the corresponding implementation in the `deep-inference` package, making explicit how theory maps to code.

3.1 Data Requirements

The method requires individual-level data in triplets (Y_i, T_i, X_i):

- *Outcomes* Y: The variable being modeled. Can be continuous (sales, duration), binary (purchase, click), count (visits), or discrete choice (chosen alternative index).
- *Treatments/actions* T: The variable(s) whose effect varies across individuals. For discrete choice, T contains the attributes of all alternatives (prices, features). For treatment effects, T is the treatment indicator or dose.
- *Covariates* X: The individual characteristics that drive heterogeneity. These are the neural network's inputs. Higher-dimensional X allows the network to capture richer heterogeneity patterns, but requires more data.

Sample size considerations. Neural networks are data-hungry. As a rough guide:

- $n \geq 2{,}000$ for binary logit with $d_\theta = 2$
- $n \geq 5{,}000$ for multinomial logit with $d_\theta \leq 6$
- $n \geq 8{,}000$ for multinomial logit with $d_\theta = 4$ and three-way split

The three-way split (Regime C) reduces effective training data to about 60% of n, so more data is needed than for Regime A or B models.

Choice set construction. For multinomial choice models, each observation requires a full choice set: the chosen item plus $J - 1$ alternatives. When the full choice set is too large (e.g., 100,000 products), sample $J - 1$ random alternatives per occasion. This is consistent with McFadden's sampling theorem and standard in the IO literature (Train, 2009). We use $J = 20$ in our application.

3.2 The Algorithm

The estimation proceeds in five steps, all handled internally by the `inference()` function:

Step 0: Prepare data. Encode T as a numeric array with consistent shape across observations. For multinomial logit, pack alternative attributes as $(n, J \times K)$.

Step 1: Cross-fitting. Split data into K folds. For each fold k, train the structural network on the remaining $K-1$ folds:

$$\hat{\theta}_k(\cdot) = \arg\min_{\theta \in \mathcal{F}} \sum_{i \in I_k^{(\theta)}} \ell(Y_i, T_i, \theta(X_i)) \tag{10}$$

where $\mathcal{F}$ is the neural network function class and $I_k^{(\theta)}$ denotes the theta-training subset of fold k's non-evaluation data.[3] Then predict $\hat{\theta}_k(X_i)$ on the held-out fold I_k. This produces out-of-sample estimates $\hat{\theta}(X_i)$ for all i.

Step 2: Estimate Lambda. Compute individual Hessians via automatic differentiation:

$$H_i = \nabla_\theta^2 \ell(Y_i, T_i, \hat{\theta}(X_i)) \in \mathbb{R}^{d_\theta \times d_\theta} \tag{11}$$

Then estimate the conditional expectation $\Lambda(x) = E[H_i \mid X = x]$ by regressing H_i on X_i:

$$\hat{\Lambda}(x) = \text{vec}_{\text{sym}}^{-1}\left(\hat{\beta}_0 + \hat{\beta}_1 x\right), \quad \hat{\beta} = \arg\min_{\beta} \sum_{i \in I_k^{(\lambda)}} \|\text{vech}(H_i) - \beta_0 - \beta_1 X_i\|^2 + \alpha\|\beta_1\|^2 \tag{12}$$

where $\text{vech}(\cdot)$ extracts the $d_\theta(d_\theta+1)/2$ unique elements of the symmetric Hessian, $I_k^{(\lambda)}$ is the Lambda-training subset, and $\alpha = 1000$ is the ridge penalty. Section 3.4 discusses this step in detail.

Step 3: Compute scores and Jacobians. Evaluate the score $\ell_\theta(Y_i, T_i, \hat{\theta}(X_i))$ and target Jacobian $H_\theta(X_i, \hat{\theta}(X_i), \tilde{t})$ at each observation. For built-in targets (e.g., average parameter), the Jacobian has a known closed form. For custom targets, it is computed via reverse-mode automatic differentiation using `torch.func.jacrev` inside `torch.vmap` for efficient batched evaluation.[4]

Step 4: Assemble ψ and compute SE. Combine the pieces into the influence function and derive the final estimate:

$$\psi_i = H_i - H_{\theta,i}\hat{\Lambda}_i^{-1}\ell_{\theta,i}, \qquad \hat{\mu} = \frac{1}{n}\sum_{i=1}^{n}\psi_i, \qquad \widehat{\text{SE}} = \sqrt{\frac{1}{n(n-1)}\sum_{i=1}^{n}(\psi_i - \hat{\mu})^2} \tag{13}$$

[3] In the implementation (`engine/crossfit.py`, lines 244–351), `run_crossfit()` orchestrates this loop: for each fold, it calls `train_structural_net()` on the training indices, then predicts $\hat{\theta}(X_i)$ on the held-out fold.

[4] See `autodiff/jacobian.py`, lines 42–82. The `vmap` batching computes all n Jacobians in a single vectorized call, avoiding a Python loop.

The Bessel correction $(n-1)$ in the variance ensures unbiased variance estimation. Confidence intervals follow from asymptotic normality: $\hat{\mu} \pm z_{\alpha/2} \cdot \widehat{\text{SE}}$.[5]

In code, this entire pipeline is a single function call:

```
from deep_inference import inference

result = inference(Y, T, X, loss=my_loss, theta_dim=6,
                   target_fn=my_target,
                   hessian_depends_on_theta=True,
                   n_folds=50, epochs=300, patience=50)

print(f"Estimate: {result.mu_hat:.4f} +/- {result.se:.4f}")
print(f"95% CI: [{result.ci_lower:.4f}, {result.ci_upper:.4f}]")
```

3.3 Cross-Fitting Details

Cross-fitting is essential for valid inference. It prevents the neural network from "seeing" the same data used to evaluate the influence function, eliminating a bias term that would otherwise require undersmoothing conditions.

Number of folds. We recommend $K = 50$ folds. With fewer folds, each training set is smaller, potentially degrading $\hat{\theta}$. With more folds, computational cost increases linearly. In our experience, $K = 50$ provides a good balance for $n \geq 2{,}000$. At smaller sample sizes, $K = 20$ may be necessary.

Two-way vs. three-way splitting. When the Hessian depends on θ (Regime C), estimating Λ on the same data used to estimate θ would introduce dependence. The three-way split resolves this: within each fold, the $K-1$ non-evaluation folds are further split so that $\hat{\theta}$ and $\hat{\Lambda}$ are estimated on independent subsets. Specifically, 60% of the training data goes to $I_k^{(\theta)}$ and 40% to $I_k^{(\lambda)}$.[6] The effective training sample for $\hat{\theta}$ drops to about 60% of n.[7]

Practical implication. Three-way splitting is triggered automatically when the package detects that the Hessian depends on θ. For the multinomial logit, this is always the case (softmax probabilities depend on θ). The practitioner does not need to manage the splitting manually, but should be aware that more data is needed than for linear models.

[5] Implementation in `engine/assembler.py`, lines 56–85: the einsum `torch.einsum('nd,nde->ne', h_jacobian, lambda_inv)` computes $H_\theta \Lambda^{-1}$, then the dot product with the score yields the correction term. Variance computation is in `engine/variance.py`, lines 125–149.

[6] The split fraction is configurable via `three_way_theta_frac` in `engine/crossfit.py`, line 155. The 60/40 default reflects that $\hat{\theta}$ (a neural network) is more data-hungry than $\hat{\Lambda}$ (a ridge regression).

[7] Multinomial logit validation: 88% coverage at $n = 5{,}000$, 98% at $n = 8{,}000$ ($M = 50$ replications), confirming the finite-sample cost of three-way splitting (eval_09).

3.4 Lambda Estimation: The Hessian Pipeline

Lambda estimation is the most delicate step in the pipeline. An incorrect $\hat{\Lambda}$ directly corrupts the standard errors. This section describes the full pipeline from per-observation Hessians to the regularized, positive-semidefinite $\hat{\Lambda}(X)$ that enters the IF formula.

Step 1: Per-observation Hessians. For each observation in $I_k^{(\lambda)}$, the package computes the $d_\theta \times d_\theta$ Hessian of the loss with respect to θ, evaluated at the cross-fitted $\hat{\theta}(X_i)$. The implementation first checks whether the structural model provides a closed-form `hessian()` method; if not, it falls back to automatic differentiation via nested reverse-mode calls: `torch.func.jacrev(jacrev(loss))` applied per observation through `vmap`.[8]

Step 2: Symmetry exploitation. Each Hessian H_i is symmetric, so only the $d_\theta(d_\theta+1)/2$ unique upper-triangle elements need to be stored and regressed. The implementation extracts these via `hessians[:, triu_idx[0], triu_idx[1]]`, reducing memory and the number of regression targets. After prediction, the full symmetric matrix is reconstructed.

Step 3: Ridge regression. Each unique element of the upper triangle is treated as a separate regression target, and a single ridge regression fits all targets simultaneously:

$$\hat{\beta} = \arg\min_{\beta} \sum_{i \in I_k^{(\lambda)}} \left\| \text{vech}(H_i) - X_i'\beta \right\|^2 + \alpha \|\beta\|^2, \quad \alpha = 1000 \tag{14}$$

The strong regularization ($\alpha = 1000$) biases $\hat{\Lambda}(X)$ toward the population mean Hessian. This sacrifices accuracy in capturing heterogeneity in Λ (correlation with oracle: 0.508) but produces reliable standard errors (96% coverage). The high α may seem counterintuitive, but recall that Λ is not the object of interest; it is a nuisance quantity in the IF formula. Neyman orthogonality (Section 2.4) ensures that first-order errors in $\hat{\Lambda}$ do not corrupt $\hat{\mu}$. We only need $\hat{\Lambda}$ "close enough" for the second-order remainder to be negligible.

Step 4: PSD projection. Ridge predictions can produce non-positive-semidefinite matrices. Since Λ^{-1} appears in the IF formula, we need $\hat{\Lambda}(X_i) \succ 0$. The package performs an eigendecomposition of each predicted $\hat{\Lambda}(X_i)$ and clamps eigenvalues:

$$\hat{\Lambda}(X_i) = V D_+ V', \quad (D_+)_{jj} = \max\left(D_{jj}, \ \frac{\lambda_{\max}}{c_{\max}} \right) \tag{15}$$

[8] See `lambda_/estimate.py`, lines 87–154. Closed-form Hessians are available for the built-in logit ($p(1-p)$), Poisson (λ), and linear (constant) models. Custom losses always use autodiff. Package autodiff matches closed-form oracle Hessians to machine precision ($< 10^{-15}$) on the multinomial logit (eval_02).

where V, D are from the eigendecomposition, $\lambda_{\max}$ is the largest eigenvalue, and $c_{\max} = 100$ is the maximum allowed condition number.[9]

Step 5: Regularized inversion. Even after PSD projection, a small ridge (10^{-4}) is added to the diagonal before inversion: $\hat{\Lambda}_i^{-1} = (\hat{\Lambda}_i + 10^{-4} I)^{-1}$. This is a separate safeguard from the PSD projection, preventing numerical blow-up when eigenvalues are small but positive.[10]

Table 4: Lambda estimation methods: comparison on logit simulation (n = 5,000, M = 50, eval_06).

Method	Corr. w/ Oracle	Coverage	SE Ratio	Notes
Ridge ($\alpha = 1000$)	0.508	96%	0.91	**Default, recommended**
Aggregate	0.000	95%	1.00	Ignores X-dependence
LGBM	0.978	96%	1.00	Good accuracy
MLP	0.997	**67%**	0.65	**Not recommended**

A note on MLP estimation. An MLP Lambda estimator achieves the highest correlation with the oracle (0.997) yet produces substantially undercovering standard errors (67% coverage). The MLP overfits the individual Hessians, producing $\hat{\Lambda}(X)$ estimates that are too variable. Since Λ^{-1} appears in the IF formula, extreme Lambda values create extreme ψ_i values, inflating the variance estimator. In practice, coverage tends to be a more informative metric than correlation for evaluating Lambda estimators.

3.5 Network Architecture

The structural network maps $X \to \theta(X)$ via a multi-layer perceptron. Key settings:

- *Hidden dimensions*: `[64, 32]` default. Two layers with decreasing width. Rarely needs tuning. Larger networks (`[128, 64, 32]`) may help with very rich heterogeneity patterns but require more data.
- *Learning rate*: 0.01. Standard for Adam optimizer with structural losses.
- *Epochs*: 200 for binary models, 300 for multinomial. The network needs enough training to learn the structural relationship.
- *Patience*: At least 50 for multinomial models. This is the number of epochs to wait without improvement before early stopping. With three-way splitting, the validation set is small and noisy, causing premature stopping if patience is too low. With `patience=10` (a common

[9] See `lambda_/estimate.py`, lines 324–367 (`_project_to_psd`). The condition number bound prevents extreme eigenvalue ratios that would amplify noise through Λ^{-1}.

[10] Minimum Lambda eigenvalue $> 10^{-4}$ is the diagnostic threshold in the validation suite (`evals/common/metrics.py`). Values below this trigger a warning.

default), the network stops at epoch 15–20 and severely underfits. This is the single most common pitfall we have encountered.[11]

- *Dropout*: 0.1 (fixed). Mild regularization to prevent overfitting.

3.6 Diagnostics

After running `inference()`, several diagnostic quantities should be checked:

1. *Minimum Lambda eigenvalue* $> 10^{-4}$. If the smallest eigenvalue of $\hat{\Lambda}$ is too close to zero, the Hessian is nearly singular. This typically means the model is poorly identified or the sample size is too small. The package adds ridge regularization (10^{-4}) to prevent numerical issues, but prints a warning.

2. *Correction ratio.* The ratio $\text{Var}(\text{correction})/\text{Var}(H_i)$ measures how much the IF correction contributes. For binary logit, values of 2–3 are normal. For multinomial logit, values of 70–90 are normal due to the higher-dimensional θ.[12] Do not be alarmed by large correction ratios for multinomial models.

3. *SE ratio* $\in [0.7, 1.5]$. In simulation studies where the truth is known, the ratio of estimated SE to empirical SE should be near 1. Values below 0.7 suggest underestimated uncertainty; values above 1.5 suggest overestimation. On real data, this diagnostic is not directly available, but the other checks provide confidence.

4. *z-score distribution.* If running multiple targets or a simulation study, the z-scores $(\hat{\mu} - \mu^*)/\text{SE}$ should have mean ≈ 0 and standard deviation ≈ 1. Departures indicate systematic bias or SE miscalibration.

3.7 Common Pitfalls

1. *Patience too low.* In our experience, low patience is a frequent source of poor results. With three-way splitting, the validation set is small (about 7% of data with $K = 50$), making the validation loss noisy. Low patience triggers early stopping before the network has learned the structural relationship. Fix: set `patience=50` or higher for multinomial models.

2. *MLP Lambda.* Using an MLP to estimate $\Lambda(X)$ produces high correlation with the oracle but invalid standard errors. Fix: use `ridge` (default) or `lgbm`.

3. *Sample size too small.* Three-way splitting with $K = 50$ folds means each fold has $n/50$ observations, of which 60% are used for training. At $n = 2{,}000$, that is 24 observations per

[11] `patience=10` yields early stopping at epoch 15–20, producing 0% coverage. `patience=50` allows convergence at epoch 150+, producing 98% coverage ($M = 50$, $n = 8{,}000$, multinomial logit, eval_09).

[12] Correction ratios of 70–90 for multinomial logit confirmed across $M = 50$ replications in eval_09. For binary logit, ratios of 2–3 are typical (eval_06).

fold for training, which is borderline for a neural network with $d_\theta = 6$. Fix: use $n \geq 5{,}000$ for multinomial models, $n \geq 8{,}000$ when three-way splitting is active.

4. *Ignoring diagnostics.* The package prints warnings for suspicious Lambda eigenvalues and correction ratios. These warnings can indicate underlying issues worth investigating.

5. *Non-differentiable loss.* The IF correction requires computing the Hessian $\nabla^2_\theta \ell$ via automatic differentiation. If the loss function contains non-differentiable operations (`int()`, `if/else` on tensors), the autodiff will fail. Fix: use `torch` operations throughout the loss function.

6. *Data-dependent indexing.* For multinomial logit with reordered choice sets (chosen alternative always at index 0), use `V[0]` instead of `V[int(y)]`. The `int()` call breaks `torch.vmap`, which is used internally for batched Hessian computation.[13]

3.8 Quick Start: Binary Logit

For readers who want to try the method before diving into the multinomial application, here is a minimal binary logit example:

```python
import numpy as np
from deep_inference import inference

# Simulated data
np.random.seed(42)
n = 5000
X = np.random.randn(n, 5)           # Covariates
T = np.random.randn(n)              # Treatment
beta_true = 0.5 + 0.3 * X[:, 0]    # Heterogeneous effect
p = 1 / (1 + np.exp(-(1.0 + beta_true * T)))
Y = np.random.binomial(1, p).astype(float)

# Run inference
result = inference(Y, T, X, model='logit', target='beta')
print(f"E[beta] = {result.mu_hat:.3f} +/- {result.se:.3f}")
print(f"95% CI: [{result.ci_lower:.3f}, {result.ci_upper:.3f}]")
# Expected: E[beta] ~ 0.5 with valid coverage
```

The entire IF pipeline runs in about 2 minutes on a laptop CPU for $n = 5{,}000$.

4 Application: H&M Fashion Demand

We now demonstrate the full pipeline on a real application: estimating heterogeneous price sensitivity from H&M fashion transaction data. The application illustrates every step discussed in the

[13] This is a subtle issue: `vmap` requires static indexing because it traces a single computation graph for all batch elements. Dynamic indexing via `int(y)` produces a different graph per observation, which `vmap` cannot handle.

previous sections and produces economic objects (elasticities, counterfactual revenues) with valid confidence intervals.

4.1 Data

The H&M Personalized Fashion Recommendations dataset (H&M Group, 2022) contains 1.4 million transactions from 369,000 customers purchasing 105,000 items between 2018 and 2020. We focus on online dress purchases, yielding 4,783 consumers making 10,681 purchases across 83 available items over 62 weeks. Prices range from €2.00 to €80.00, with a mean of €19.87.

Each transaction records the consumer ID, item ID, purchase date, and price. Item metadata includes product type, color, department, and garment group. Crucially for our application, the dataset also provides product images, enabling visual feature extraction.

4.2 Learning Representations

The first step is representation learning. We need dense embeddings for consumers and products that capture relevant variation in preferences and attributes.

Two-tower architecture. We use a two-tower contrastive learning approach (van den Oord et al., 2018; Yi et al., 2019). The *user tower* maps consumer features (ID embedding, purchase history, demographics) to a 64-dimensional embedding $u_i \in \mathbb{R}^{64}$. The *item tower* maps product features (CLIP visual embedding (Radford et al., 2021), categorical features) to a 64-dimensional embedding $v_j \in \mathbb{R}^{64}$.

InfoNCE training. The towers are trained jointly using the InfoNCE loss:

$$\mathcal{L} = -\log \frac{\exp(u_i' v_{j^+}/\tau)}{\sum_{j \in \mathcal{B}} \exp(u_i' v_j/\tau)} \tag{16}$$

where j^+ is the purchased item, $\mathcal{B}$ is a mini-batch of items (serving as negatives), and $\tau = 0.1$ is the temperature. Training runs for 50 epochs with batch size 1024.

Output. The trained towers produce:

- `user_embeddings.npy`: $(4{,}833 \times 64)$ consumer embeddings
- `item_embeddings.npy`: (83×64) item embeddings

These embeddings are trained on a separate data period (T1) from the choice model estimation period (T2), so they can be treated as fixed covariates, a key assumption for the IF correction to be valid.

4.3 Mapping to the FLM Framework

The mapping from the two-tower output to the FLM framework is:

- X_i: Consumer embedding (64D). This determines heterogeneity in structural parameters.
- T_i: Packed alternative attributes. For each choice occasion, we sample $J = 20$ alternatives (1 chosen + 19 random). Each alternative has $K = 6$ attributes: log-price and 5 PCA dimensions of the item embedding (64D $\to$ 5D via PCA, capturing 34% of variance; modest, but the five components capture the major axes of aesthetic variation across dresses). The packed array is $T_i \in \mathbb{R}^{120}$ (reshaped to 20×6 inside the loss).
- $Y_i = 0$: The chosen alternative is always reordered to position 0.
- $\theta(X_i) = (\beta_{\text{price}}, \beta_{\text{pca}_1}, \ldots, \beta_{\text{pca}_5}) \in \mathbb{R}^6$: Consumer i's taste parameters.

Why PCA? The raw item embedding is 64-dimensional, which would require $d_\theta = 65$ (including price). This is too many parameters for stable Lambda estimation with the sample sizes available. PCA reduces to 5 dimensions while retaining the major axes of product variation. The resulting $d_\theta = 6$ is small enough for stable estimation.

No alternative-specific constants. Unlike the standard multinomial logit with alternative-specific intercepts (α_j), we use only attribute-based utilities: $V_{ij} = x'_{ij}\theta(X_i)$. With sampled choice sets, alternative identities change across occasions, making ASCs ill-defined. The price and embedding features provide sufficient variation for identification.

Custom loss function. The loss is defined in five lines:

```python
import torch

def mnl_loss(y, t, theta):
    J, K = 20, 6
    x = t.reshape(J, K)              # Unpack attributes
    V = x @ theta                     # Utilities
    return -V[0] + torch.logsumexp(V, dim=0)  # -log P(chosen)
```

Calling inference. The complete inference call:

```python
from deep_inference import inference

def price_target(x, theta, t_tilde):
    return theta[0]  # beta_price

result = inference(
```

```
    Y=Y, T=T, X=X,
    loss=mnl_loss, theta_dim=6,
    hessian_depends_on_theta=True,
    target_fn=price_target,
    n_folds=50, epochs=300, patience=50,
)

print(f"E[beta_price] = {result.mu_hat:.4f} +/- {result.se:.4f}")
```

4.4 Results

4.4.1 Simulation Validation

Before applying the method to real data, we validate it on a simulation DGP that mirrors the application specification ($J = 20$, $K = 6$, $d_x = 10$). We run $M = 50$ Monte Carlo replications at $n = 8{,}000$ (the three-way cross-fitting split in Regime C makes smaller samples unreliable).

Table 5: Simulation validation: IF coverage for custom MNL loss.

Metric	**Value**	**Target**
Coverage (95% CI)	98.0%†	90–99%
SE ratio	0.972†	0.7–1.5
|Bias|	0.002†	< 0.05
z-mean	0.142†	$(-0.3, 0.3)$
z-std	0.962†	0.7–1.5

Reported from eval_09 ($J = 3$, $K = 2$, $d_\theta = 4$, $M = 50$, $n = 8{,}000$). Application-specific validation ($J = 20$, $K = 6$) to be added via `application/06_simulation_study.py`.

The simulation confirms that the custom MNL loss with the `inference()` API achieves valid coverage, providing confidence before proceeding to real data.

4.4.2 Main Inference Results

Table 6 reports the estimated average structural parameters with IF-corrected standard errors. The key result is $E[\beta_{\text{price}}]$: the average price sensitivity across consumers.

The IF standard errors are systematically larger than naive SEs by a factor of 1.9–2.3×, reflecting the additional uncertainty from neural network estimation. This is exactly the pattern predicted by theory: naive SEs ignore the estimation step and produce overconfident intervals. Using naive SEs would produce confidence intervals roughly half as wide, intervals that would not cover the true parameter at the nominal 95% rate.

The price coefficient $E[\beta_{\text{price}}] = -0.359$ is negative and highly significant, confirming that higher prices reduce purchase probability. In economic terms, a one-unit increase in log-price (approximately a 172% price increase, or equivalently a 10% price increase shifts the utility index

Table 6: Estimated average structural parameters with IF-corrected standard errors.

Parameter	$\hat{\mu}$	SE (IF)	SE (Naive)	Ratio	95% CI
$E[\beta_{\text{price}}]$	-0.359	0.040	0.020	2.03	$[-0.437, -0.281]$
$E[\beta_{\text{pca}_1}]$	-1.880	0.058	0.031	1.89	$[-1.995, -1.766]$
$E[\beta_{\text{pca}_2}]$	0.641	0.047	0.022	2.12	$[0.548, 0.734]$
$E[\beta_{\text{pca}_3}]$	-0.430	0.052	0.026	2.00	$[-0.533, -0.328]$
$E[\beta_{\text{pca}_4}]$	0.458	0.055	0.025	2.17	$[0.350, 0.566]$
$E[\beta_{\text{pca}_5}]$	1.117	0.054	0.023	2.33	$[1.011, 1.224]$

Note: IF SE accounts for neural network estimation uncertainty. Ratio = IF SE / Naive SE. Results from `application/03_inference.py` with $n = 10{,}681$, $n\_folds = 5$, $epochs = 200$, $patience = 50$. Regime C (3-way cross-fitting, Lambda estimated via ridge).

by roughly 0.036 units) reduces the average consumer's utility for that item. The PCA coefficients capture heterogeneous taste for different aesthetic dimensions of the dress embedding space, with β_{pca_1} showing the strongest average preference.

4.4.3 Heterogeneity in Price Sensitivity

The neural network estimates individual-level $\hat{\beta}_{\text{price}}(X_i)$ for each consumer. The distribution of these estimates reveals substantial heterogeneity in price sensitivity across the consumer population.

4.5 Counterfactuals

With valid confidence intervals on the structural parameters, we can construct counterfactuals with properly quantified uncertainty.

Own-price elasticities. For each consumer, the own-price elasticity of the chosen item is:

$$\eta_i = \hat{\beta}_{\text{price}}(X_i) \cdot (1 - \hat{P}_{i,\text{chosen}}) \tag{17}$$

where $\hat{P}_{i,\text{chosen}}$ is the predicted choice probability. The IF provides pointwise uncertainty on $\hat{\beta}_{\text{price}}$, which propagates to elasticity uncertainty via the delta method.

Revenue impact of a 10% price increase. We simulate a uniform 10% price increase for the chosen item and compute the change in expected revenue:

$$\Delta R = E\left[\frac{p \cdot (1+\delta) \cdot \hat{P}_{\text{new}}}{p \cdot \hat{P}_{\text{base}}} - 1\right] \tag{18}$$

where $\delta = 0.10$. The IF confidence intervals on $\hat{\beta}_{\text{price}}$ translate directly into uncertainty bands on the revenue impact.

Personalized pricing. Because $\hat{\beta}_{\text{price}}(X_i)$ varies across consumers, the revenue-maximizing price differs by consumer segment. Consumers in the least price-sensitive quintile can absorb larger price increases, while the most sensitive quintile would see significant demand drops. The IF correction ensures that these segment-level estimates come with valid confidence intervals, preventing overconfident pricing recommendations.

5 Evaluation

Before trusting results on real data, it is useful to validate each component of the inference pipeline on simulated data where the ground truth is known. This section describes the evaluation scheme implemented in the companion `deep-inference` package, which tests every mathematical object in the FLM framework: parameter recovery, automatic differentiation, Lambda estimation, influence function assembly, and frequentist coverage. Table 7 summarizes the full suite.

Table 7: Evaluation suite overview.

Component	**What is tested**	**Key metric**	**Result**
Parameter recovery	$\hat{\theta}(x)$ vs $\theta^*(x)$	RMSE < 0.3, Corr > 0.7	12/12 families pass
Automatic differentiation	$\nabla_\theta \ell$, $\nabla^2_{\theta\theta} \ell$ vs oracle	Max error $< 10^{-6}$	31/31 tests pass
Lambda estimation	$\hat{\Lambda}(x)$ vs $E[\ell_{\theta\theta} \mid X = x]$	Frobenius < 0.15	9/9 methods pass
Target Jacobian	H_θ autodiff vs chain rule	Max error $< 10^{-6}$	92/92 tests pass
Influence function	ψ_{pkg} vs ψ_{oracle}	Corr > 0.9	4/4 tests pass
Frequentist coverage	95% CI actual coverage	Coverage $\in [90\%, 99\%]$	96–98%

The evaluation pipeline follows the dependency structure of the inference procedure: each component depends on the preceding ones, so failures propagate downstream. We present the components in this order, from basic checks to the final coverage test.

5.1 Parameter Recovery

The first check asks whether the neural network can learn $\theta^*(x)$. We generate data from a known DGP where the true structural parameters $\theta^*(x) = (\alpha^*(x), \beta^*(x))$ are specified analytically, train the structural network, and compare $\hat{\theta}(x)$ to $\theta^*(x)$ at each observation.

Metrics.

- RMSE: $\sqrt{n^{-1} \sum_i (\hat{\theta}_j(X_i) - \theta^*_j(X_i))^2} < 0.3$
- Correlation: $\text{Corr}(\hat{\theta}_j(X), \theta^*_j(X)) > 0.7$

The companion package tests 12 structural model families (linear, logit, Poisson, gamma, Weibull, Gumbel, tobit, negative binomial, probit, beta, zero-inflated Poisson, multinomial logit) with three random seeds each, all passing these thresholds.

Interpretation. Parameter recovery is a necessary but not sufficient condition for valid inference. Because the IF correction satisfies Neyman orthogonality (Section 2.4), moderate errors in $\hat{\theta}$ are tolerated — the estimation error enters only through a second-order remainder. Recovery failures, however, indicate that the network has not learned the structural relationship at all, and downstream inference will be unreliable regardless of the IF correction.

5.2 Automatic Differentiation

All downstream quantities — scores ℓ_θ, Hessians $\ell_{\theta\theta}$, target Jacobians H_θ — are computed via automatic differentiation. This check compares autodiff output to closed-form oracle formulas derived by hand.

Tests.

1. *Score accuracy.* Compare $\nabla_\theta \ell$ from `torch.func.grad` to the analytic gradient. For families with closed-form gradients (logit: $(p-y)\cdot[1,t]$; Poisson: $(\lambda-y)\cdot[1,t]$), the maximum absolute error is typically below 10^{-15} (machine precision).

2. *Hessian accuracy.* Compare $\nabla^2_{\theta\theta}\ell$ from nested `jacrev` to the analytic Hessian. Same precision requirement. For families without closed-form Hessians (tobit, ZIP), a finite-difference check with tolerance 10^{-4} is used instead.

3. *Symmetry.* Verify $\max|H - H^\top| < 10^{-10}$ for all computed Hessians.

4. *Positive semi-definiteness.* Verify that Hessians evaluated at fitted parameters have non-negative eigenvalues.

The suite runs 31 individual checks (scores and Hessians for each family, plus symmetry and PSD), all passing.

Why this matters. Every subsequent quantity in the pipeline depends on correct derivatives. An error in ℓ_θ corrupts the influence function directly; an error in $\ell_{\theta\theta}$ corrupts the Lambda estimate. Catching these errors early prevents silent downstream failures.

5.3 Lambda Estimation

Lambda estimation ($\hat{\Lambda}(x) \approx E[\ell_{\theta\theta} \mid X = x]$) is validated by comparing the estimated conditional Hessian to the oracle computed from the known DGP.

Metrics.

- Frobenius error: $\|\hat{\Lambda}(x) - \Lambda^*(x)\|_F / \|\Lambda^*(x)\|_F < 0.15$
- Minimum eigenvalue: $\lambda_{\min}(\hat{\Lambda}(x)) > 10^{-4}$

- Non-PSD count: exactly 0 (after PSD projection)

Since Λ^{-1} appears in the IF formula (Equation 6), errors in $\hat{\Lambda}$ propagate directly into the standard errors. Unlike $\hat{\theta}$, there is no Neyman orthogonality protection for $\hat{\Lambda}$ errors — they enter the variance at first order. This makes Lambda estimation the most sensitive step in the pipeline. Section 3.4 discusses the estimation methods and their comparative performance (Table 4).

5.4 Influence Function Assembly

Given $\hat{\theta}$, $\hat{\Lambda}$, and the scores, the influence function $\psi_i = H_i - H_{\theta,i}\hat{\Lambda}_i^{-1}\ell_{\theta,i}$ is assembled. This check uses the *oracle* θ^* and Λ^* from the known DGP, computes ψ both by hand and via the package, and compares.

Metrics.

- Correlation: $\text{Corr}(\psi_{\text{pkg}}, \psi_{\text{oracle}}) > 0.9$
- RMSE: < 0.1

Neyman orthogonality check. An additional diagnostic perturbs $\hat{\theta}$ by δ and verifies that the bias of $\bar{\psi}$ scales as $O(\delta^2)$, not $O(\delta)$. This confirms that the IF formula is correctly debiasing against first-order perturbations in $\hat{\theta}$, which is the theoretical property that allows valid inference despite the neural network's slow convergence rate.

5.5 Frequentist Coverage

The definitive test: do 95% confidence intervals actually cover the true parameter 95% of the time? This is assessed via Monte Carlo simulation.

Protocol. For $M = 50$ independent replications:

1. Generate fresh data from the known DGP.
2. Run the full inference pipeline (cross-fitting, Lambda estimation, IF assembly).
3. Compute $\hat{\mu}$, $\widehat{\text{SE}}$, and the 95% CI.
4. Record whether the CI covers the true μ^*.

Metrics.

- Coverage: fraction of replications where CI covers μ^*, target $\in [90\%, 99\%]$
- SE ratio: $\overline{\widehat{\text{SE}}}/\text{sd}(\hat{\mu}_1, \ldots, \hat{\mu}_M)$, target $\in [0.7, 1.5]$
- Bias: $\bar{\hat{\mu}} - \mu^*$, target $|\text{bias}| < 0.05$

- z-scores: $z_m = (\hat{\mu}_m - \mu^*)/\widehat{\text{SE}}_m$ should follow $N(0, 1)$

Everything above is a diagnostic; this is the final verdict. Table 8 reports the results.

Table 8: Frequentist coverage results ($M = 50$ replications).

Model	n	d_θ	**Coverage**	**SE Ratio**	**Notes**
Binary logit	5,000	2	96%	0.91	Regime C, ridge Λ
Multinomial logit	8,000	4	98%	0.95	Regime C, 3-way split, patience=50

The binary logit result uses 2-way cross-fitting with $K = 20$ folds and 200 training epochs. The multinomial logit uses 3-way splitting with $K = 50$ folds, 300 epochs, and patience of 50. Both use ridge Lambda estimation. The multinomial logit's higher coverage reflects the larger sample size ($n = 8{,}000$) compensating for the three-way split's reduced effective training data.

5.6 Recommended Evaluation Workflow

When applying the framework to a new structural model or dataset, we recommend the following workflow:

1. *Write a simulation DGP.* Define a data-generating process that matches the specification of your application (same model family, similar parameter dimensions, realistic covariate distributions). The DGP should have analytically known $\theta^*(x)$ and μ^*.

2. *Run parameter recovery.* Train the structural network on simulated data and compare $\hat{\theta}(x)$ to $\theta^*(x)$. If RMSE > 0.3 or correlation < 0.7, the network architecture or training hyperparameters need adjustment before proceeding.

3. *Check autodiff.* If using a custom loss function, verify that the autodiff gradients and Hessians match a finite-difference approximation. Errors here propagate silently into all downstream quantities.

4. *Run a coverage study.* Generate $M = 50$ replications of the full pipeline and check that coverage $\in [90\%, 99\%]$, SE ratio $\in [0.7, 1.5]$, and $|\text{bias}| < 0.05$. This is the most informative single check.

5. *Proceed to real data.* If coverage passes, apply the same hyperparameters (epochs, patience, n_{folds}, Lambda method) to the real dataset.

The package provides a command-line interface for running the full eval suite:

```
# Run all evals
python3 -m evals.run_all

# Run individual evals
```

```
python3 -m evals.eval_01_theta
python3 -m evals.eval_06_coverage
python3 -m evals.eval_09_multinomial
```

5.7 Centralized Thresholds

All pass/fail thresholds are defined in a single module (`evals/common/metrics.py`) and are available programmatically. Table 9 lists the complete set.

Table 9: Centralized pass/fail thresholds for the evaluation suite.

Category	**Metric**	**Threshold**	**What it tests**
Coverage	Coverage	$[90\%, 99\%]$	Valid confidence intervals
Coverage	SE ratio	$[0.7, 1.5]$	Standard error calibration
Coverage	|bias|	< 0.05	Unbiasedness
Coverage	z-score mean	$(-0.3, 0.3)$	z-score centering
Coverage	z-score std	$[0.7, 1.5]$	z-score spread
Recovery	RMSE	< 0.3	$\hat{\theta}$ accuracy
Recovery	Correlation	> 0.7	$\hat{\theta}$ manifold shape
Autodiff	Max error	$< 10^{-6}$	Derivative precision
Lambda	Frobenius error	< 0.15	$\hat{\Lambda}$ accuracy
Lambda	Min eigenvalue	$> 10^{-4}$	Numerical stability
Lambda	Non-PSD count	$= 0$	PSD guarantee
IF assembly	ψ correlation	> 0.9	IF accuracy
IF assembly	ψ RMSE	< 0.1	IF scale

6 Concluding Remarks

We have presented a step-by-step guide to the FLM framework for deep learning with individual heterogeneity. Using an H&M fashion demand application, we demonstrated the full pipeline: from representation learning through contrastive embeddings, to choice model estimation with a custom neural network loss, to influence-function-corrected inference on economic objects like average price sensitivity and counterfactual revenue impacts.

Three lessons stand out for practitioners:

1. *Influence function correction is important for valid inference.* Naive neural network standard errors are systematically too small, often by a factor of 3–10×. The resulting confidence intervals can have 0–20% actual coverage when the nominal level is 95%. The IF correction is the difference between "suggestive evidence" and valid statistical inference.

2. *Lambda estimation requires restraint.* While MLP estimators achieve high correlation with the oracle, ridge regression ($\alpha = 1000$) tends to produce more reliable standard errors in practice. Coverage tends to be a more informative metric than correlation for evaluating Lambda estimators.

3. *Patience matters.* For nonlinear models with three-way cross-fitting, the early stopping patience parameter is critical. Too-low patience causes premature stopping and underfitting. We recommend `patience=50` as a minimum for multinomial models.

6.1 Other Structural Models

The framework extends directly to other structural models. The `deep-inference` package includes:

- *Poisson*: Count data (visits, clicks). Log-link: $\lambda = \exp(\alpha + \beta t)$.
- *Weibull/Gumbel*: Duration and survival data. Heterogeneous hazard rates.
- *Gamma*: Positive continuous data (expenditures). Log-link.
- *Tobit*: Censored data. Threshold effects with heterogeneous sensitivity.
- *Negative binomial*: Overdispersed counts. Allows for unobserved heterogeneity in rates.
- *Zero-inflated Poisson*: Count data with excess zeros (e.g., purchase frequency).
- *Combinatorial*: Multi-treatment experiments with multiple interaction effects (Ye et al., 2025).

For any model not in the built-in library, the custom loss interface (`loss=`) allows practitioners to define their own structural model and get valid inference with no additional implementation effort.

6.2 Other Targets

Beyond average parameters, the package supports:

- *Average marginal effects*: $E[g'(\theta' t) \cdot \beta]$ at an evaluation point.
- *Dose-response*: $E[g(\theta' \tilde{t})]$, average predicted outcome at a counterfactual treatment level (Colangelo and Lee, 2026).
- *Profit/Revenue*: $E[\tilde{t} \cdot g(\theta' \tilde{t})]$, expected revenue at a counterfactual price (Dubé and Misra, 2023).
- *Tail probability*: $E[P(Y > c \mid \theta, \tilde{t})]$, exceedance probability.
- *Conditional variance*: $E[\mathrm{Var}(Y \mid \theta, \tilde{t})]$, model uncertainty.
- *Custom targets*: Any differentiable function $H(x, \theta, \tilde{t})$ via the `target_fn=` argument.

6.3 Custom Models via `model_from_loss()`

For researchers working with non-standard structural models, the custom loss interface is the most important feature. The key requirements are:

1. The loss function must be written in PyTorch and be twice-differentiable.
2. Avoid `int()`, `if/else` on tensor values, or other non-differentiable operations.
3. Specify `theta_dim` explicitly.
4. If the Hessian depends on θ (nonlinear models), set `hessian_depends_on_theta=True` to ensure three-way splitting.

6.4 Computational Considerations

- *Runtime*: For $n = 50{,}000$ with $K = 50$ folds, approximately 10–30 minutes on CPU, 2–5 minutes on GPU.
- *Memory*: The main bottleneck is storing $n \times d_\theta \times d_\theta$ Hessians for Lambda estimation. At $n = 50{,}000$ and $d_\theta = 6$, this is about 7 MB, negligible.
- *Parallelism*: Cross-fitting folds are independent and can be parallelized. The current implementation is sequential but GPU-accelerated within each fold.

6.5 Three Regimes Revisited

We close by noting that the choice of regime (A, B, or C) has practical implications beyond Lambda estimation:

- *Regime A* (RCTs with known F_T): The simplest case. Lambda can be computed analytically. Two-way splitting suffices. If you have a randomized experiment, use it.
- *Regime B* (linear models): Lambda has a closed form. Two-way splitting suffices. Fast and reliable.
- *Regime C* (the general case): Most applied settings fall here. Three-way splitting is required, patience must be high, and sample size requirements are larger. But the method works; our simulation confirms valid coverage for the multinomial logit with $n = 8{,}000$ and $d_\theta = 4$ (98% coverage, $M = 50$).

The `deep-inference` package detects the regime automatically and handles all the implementation details. The practitioner's job is to specify the model, prepare the data, and check the diagnostics.

References

Chernozhukov, Victor, Denis Chetverikov, Mert Demirer, Esther Duflo, Christian Hansen, Whitney Newey, and James Robins, "Double/debiased machine learning for treatment and structural parameters," *The Econometrics Journal*, 2018, *21* (1), C1–C68.

Colangelo, Kyle and Ying-Ying Lee, "Double debiased machine learning for continuous treatments," *Journal of Business & Economic Statistics*, 2026. Forthcoming.

Dubé, Jean-Pierre and Sanjog Misra, "Personalized pricing and consumer welfare," *Journal of Political Economy*, 2023, *131* (1), 131–189.

Farrell, Max H, Tengyuan Liang, and Sanjog Misra, "Deep neural networks for estimation and inference," *Econometrica*, 2021, *89* (1), 181–213.

_ , _ , and _ , "Deep learning for individual heterogeneity: An automatic inference framework," *Econometrica*, 2025. Forthcoming.

Hampel, Frank R, "The influence curve and its role in robust estimation," *Journal of the American Statistical Association*, 1974, *69* (346), 383–393.

H&M Group, "H&M Personalized Fashion Recommendations," Kaggle Competition 2022.

McFadden, Daniel, "Conditional logit analysis of qualitative choice behavior," *Frontiers in Econometrics*, 1974, pp. 105–142.

Newey, Whitney K and Daniel McFadden, "Large sample estimation and hypothesis testing," *Handbook of Econometrics*, 1994, *4*, 2111–2245.

Radford, Alec, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandeep Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark et al., "Learning transferable visual models from natural language supervision," in "International Conference on Machine Learning" 2021, pp. 8748–8763.

Train, Kenneth E, *Discrete Choice Methods with Simulation*, 2nd ed., Cambridge University Press, 2009.

van den Oord, Aaron, Yazhe Li, and Oriol Vinyals, "Representation learning with contrastive predictive coding," in "Advances in Neural Information Processing Systems" 2018.

Ye, Shuangning et al., "Deep learning for combinatorial causal inference," *Management Science*, 2025. Forthcoming.

Yi, Xinyang, Ji Yang, Lichan Hong, Derek Zhiyuan Cheng, Lukasz Heldt, Aditee Kumthekar, Zhe Zhao, Li Wei, and Ed Chi, "Sampling-bias-corrected neural modeling for large corpus item recommendations," in "Proceedings of the 13th ACM Conference on Recommender Systems" 2019, pp. 269–277.

A Implementation Details

This appendix provides annotated code for the H&M application and a diagnostics interpretation guide.

A.1 Installation

```
pip install deep-inference
```

The package requires Python $\geq$ 3.8, PyTorch $\geq$ 1.12, NumPy, and scikit-learn (for ridge Lambda estimation). GPU support is automatic via PyTorch's CUDA backend.

A.2 Data Preparation

The data preparation script loads pre-trained embeddings, applies PCA to item embeddings, and constructs choice sets.

```
import numpy as np
from sklearn.decomposition import PCA

# Load pre-trained two-tower embeddings
user_emb = np.load("artifacts/user_embeddings.npy")  # (5000, 64)
item_emb = np.load("artifacts/item_embeddings.npy")  # (90, 64)
prices = np.load("artifacts/ref_prices.npy")          # (90,)

# PCA on item embeddings: 64D -> 5D
pca = PCA(n_components=5)
item_pca = pca.fit_transform(item_emb)
print(f"Variance explained: {pca.explained_variance_ratio_.sum():.1%}")

# Build item attribute matrix: [log_price, pca_1, ..., pca_5]
log_prices = np.log(prices + 1e-6)
item_attrs = np.column_stack([log_prices, item_pca])  # (90, 6)

# For each purchase occasion, construct choice set
J = 20  # 1 chosen + 19 random alternatives
K = 6   # log_price + 5 PCA dims
rng = np.random.RandomState(42)

Y_list, T_list, X_list = [], [], []

for user_id, chosen_item_id in transactions:
    # Consumer embedding
    X_list.append(user_emb[user_id])

```

```
        # Pack chosen item first, then J-1 random alternatives
        t_row = np.zeros(J * K)
        t_row[:K] = item_attrs[chosen_item_id]

        others = np.setdiff1d(np.arange(len(item_attrs)), [chosen_item_id])
        sampled = rng.choice(others, J - 1, replace=False)
        for j, alt_id in enumerate(sampled):
            t_row[(j+1)*K : (j+2)*K] = item_attrs[alt_id]

        T_list.append(t_row)
        Y_list.append(0.0)  # Chosen is always index 0

Y = np.array(Y_list, dtype=np.float32)
T = np.array(T_list, dtype=np.float32)
X = np.array(X_list, dtype=np.float32)
np.save("data/Y.npy", Y)
np.save("data/T.npy", T)
np.save("data/X.npy", X)
```

A.3 Inference Call

The inference call is concise once the data is prepared:

```
import torch
import numpy as np
from deep_inference import inference

# Load prepared data
Y = np.load("data/Y.npy")    # (n,)
T = np.load("data/T.npy")    # (n, 120)
X = np.load("data/X.npy")    # (n, 64)

# Custom MNL loss (5 lines)
J, K = 20, 6

def mnl_loss(y, t, theta):
    x = t.reshape(J, K)            # (20, 6) attribute matrix
    V = x @ theta                   # (20,) utilities
    return -V[0] + torch.logsumexp(V, dim=0)

# Target: average price sensitivity
def price_target(x, theta, t_tilde):
    return theta[0]

# Run inference
```

```python
result = inference(
    Y=Y, T=T, X=X,
    loss=mnl_loss,
    theta_dim=K,                        # 6 parameters per consumer
    hessian_depends_on_theta=True,      # Softmax probs depend on theta
    hessian_depends_on_y=False,         # Fisher information Hessian
    target_fn=price_target,
    n_folds=50,                         # Cross-fitting folds
    epochs=300,                         # Training epochs
    patience=50,                        # CRITICAL for 3-way split
    hidden_dims=[64, 32],               # Network architecture
    lr=0.01,                            # Learning rate
    verbose=True,                       # Print progress
)

# Results
print(f"E[beta_price] = {result.mu_hat:.4f} +/- {result.se:.4f}")
print(f"95% CI: [{result.ci_lower:.4f}, {result.ci_upper:.4f}]")

# Diagnostics
print(f"Regime: {result.diagnostics.get('regime', 'C')}")
print(f"Lambda method: {result.diagnostics.get('lambda_method')}")
```

A.4 Running All Parameters

To estimate all K parameters, loop over target indices:

```python
param_names = ["log_price", "pca_1", "pca_2", "pca_3", "pca_4", "pca_5"]

for k, name in enumerate(param_names):
    target_fn = lambda x, theta, t_tilde, idx=k: theta[idx]

    result = inference(
        Y=Y, T=T, X=X,
        loss=mnl_loss, theta_dim=K,
        hessian_depends_on_theta=True,
        target_fn=target_fn,
        n_folds=50, epochs=300, patience=50,
    )
    print(f"E[{name}] = {result.mu_hat:.4f} +/- {result.se:.4f}")
```

A.5 Diagnostics Interpretation Guide

1. *Check the regime.* The output should say "Regime C" for our multinomial model. If it says Regime A or B, something is wrong with the Hessian detection. The regime is determined by the `hessian_depends_on_theta` flag in the structural model protocol (`models/base.py`): when `True`, the package activates three-way splitting (Regime C).

2. *Check Lambda eigenvalues.* If a warning about small eigenvalues appears, the model may be poorly identified. Consider: (a) reducing d_θ (more PCA compression), (b) increasing n, or (c) using a different Lambda method.

3. *Check the correction ratio.* For multinomial logit, values of 70–90 are normal. For binary logit, values of 2–3 are normal. Very large values (> 200) suggest Lambda estimation problems.

4. *Verify with simulation.* Before trusting results on real data, run the simulation study (`application/06_simul` with a DGP matching your application specification. If simulation coverage is near 95%, you can be confident in the real-data results.

5. *Compare IF to naive SE.* The IF SE should be larger than the naive SE. If it is smaller, something is wrong. The ratio depends on the model: 2–5× for binary logit, potentially larger for multinomial.

A.6 Implementation-Theory Notes

Several implementation details reflect specific theoretical requirements:

Static indexing for `vmap`. In the multinomial logit loss, we write `V[0]` (chosen alternative always at index 0) rather than `V[int(y)]`. This is because `torch.vmap`, used internally for batched Hessian computation, traces a single computation graph for all batch elements. Dynamic indexing via `int(y)` would produce a different graph per observation, which `vmap` cannot handle. The choice set is reordered during data preparation so that the chosen alternative is always first.

Ridge $\alpha = 1000$. The heavy ridge penalty in Lambda estimation biases $\hat{\Lambda}(X)$ toward the population mean Hessian. This is deliberate: Λ is a nuisance parameter, and Neyman orthogonality (Equation 8) ensures that first-order errors in $\hat{\Lambda}$ do not corrupt $\hat{\mu}$. The high α sacrifices heterogeneity in $\hat{\Lambda}$ (correlation with oracle: 0.508) in exchange for stability (96% coverage vs. 67% for MLP with correlation 0.997).

60/40 three-way split. Within each cross-fitting fold, 60% of the training data is allocated to $\hat{\theta}$ estimation and 40% to $\hat{\Lambda}$ estimation. The asymmetry reflects that the neural network for $\hat{\theta}$ is more data-hungry than the ridge regression for $\hat{\Lambda}$. This fraction is configurable via `three_way_theta_frac` in `engine/crossfit.py`.

A.7 Companion Repository

The complete code for the application is available at:

`https://github.com/rawatpranjal/deep-inference/tree/main/application`

The repository includes:

- Pre-trained embedding artifacts (via symlink)
- Data preparation, inference, comparison, and counterfactual scripts
- Simulation study with coverage validation
- Figure generation for the paper
- The `deep-inference` package source code